

CSOPESY -- 2.1 Process IPC

In continuation from [CSOPESY -- 2 Processes](#)

Linux - a program should only do one thing, according to Linus Torvalds, the creator of Linux

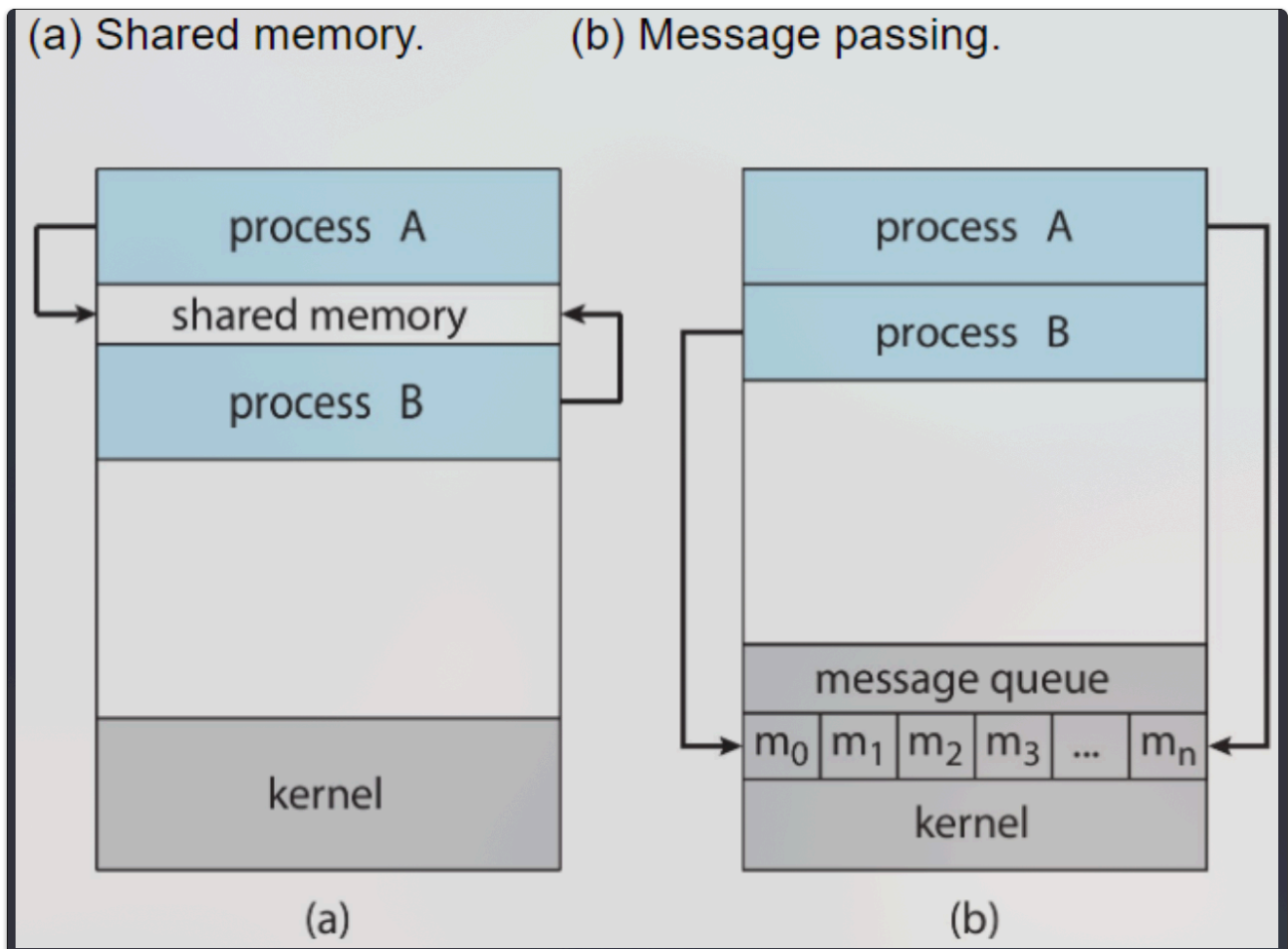


Image 1: The two ways to for processes to communicate

- **Shared memory** is where two processes would be able to share variables
 - declare variables, then declare them as shared
 - nuances:
 - kind of hard to do this especially if not used in coding threaded applications
 - technically when you share memory, ideally the developer would implement a form of data synchronization
 - Example: If you have a pdf file open then try to delete it from file explorer, it will throw an error
 - The operating system locked the particular file because its being used by something else
 - some compilers naman have a tendency to "optimize" a particular variable
 - Remember in ARCH2 days, before you can manipulate something, that something is stored in the register. Shared memory is in register. So whats

going to happen, a compiler will now keep doing whatever code only in the register not in CPU (i.e., no writeback after fetch decode execute)...

- declaring something as volatile will allow then for writebacks because it will now listen for any update on the variable

- **Message Passing** where processes will just pass message instead of sharing a variable
 - will use facility of OS to pass messages to each other
 - Example: CSNETWK made us use TCP/IP Stack by the OS, we were able to connect multiple sockets and processes
 - nuances:
 - overhead
 - however, dev need not to implement cause system calls are already provided by OS.

Producer-Consumer Problem

- technically any message passing becomes a producer-consumer problem
 - producer process creates info, consumer process consumes
- 2 variations:
 - **unbounded buffer** - places no practical limit on size of buffer
 - producer never waits
 - consumer waits if no buffer to consume
 - Example: Sir will not wait until we can digest the lesson.
 - **bounded buffer** - there is feedback loop, assumes theres a fixed buffer size
 - bucket is full, therefore cannot put more water
 - producer must wait if all buffers are full
 - consumer waits if no buffer to consume

IPC - Shared memory

- developers must have the skill to do this
- Communication is under control of users processes not OS, so technically under dev

Full Buffer

- A possible solution to consumer-producer problem that fills all buffers
 - integer count that keeps track of full buffers
 - counter set to 0
 - integer count incremented by producer after produces a new buffer
 - integer count is decremented by consumer after consuming a buffer

IPC Message Passing

- Process communicate each other without using shared variables
- IPC facility provides two operations
 - `send(message)`
 - `receive(message)`
- Message size is either fixed or variable

Time its fixed: fixed by virtue of number of bits in the bus

Time its variable: TCP has a default max of 1500, but if your message is shorter than that, then it will change to that.

If processes P and Q wish to communicate, they need to:

- establish a communication link between them
 - Sir I want to talk to you, I have to call his attention
 - If not mutual, I'm saying my message but sir is not gonna receive it.
 - that is why two base operations are `send(message)` and `receive(message)`
- exchange messages via send/receive

Possible issues in implementation:

- How are links established?
- Can a link be associated with more than two processes?
- Is a link unidirectional or bi-directional?
- Capacity of a link?

Chrome is memory heavy rin kasi it spawns multiple threads just to display a web page

Implementations of a Communication Link

1. Physical

- shared memory
- hardware bus
 - yung parang naglalag yung keyboard and mouse if youre copying something from a hard drive
 - USB (universal serial bus) is shared bus. so now mouse and kb is contending with the time slice to use the bus.
- network

2. Logical

- direct or indirect
- sync or asynch
- automatic or explicit

Direct Communication

- Processes name each other explicitly
 - send (P, message) - send a message to process P
 - EVAN DE GUZMAN, AWP MID DONT PEEK!!!!
 - receive (Q, message) - receive a message from process Q
 - I, EVAN DE GUZMAN, have received your message o dear teammate of mine
 - send(Q, message)
 - HOWEVER I WILL EGO PEEK!!!
- Properties of communication link
 - links are establish automatically
 - A link is associated with exactly ONE pair of communicating processes
 - usually bi-directional

Indirect Communication

- Messages are directed and received from mailboxes (ports)
 - Each mailbox has unique ID
 - processes can communicate only if they share a mailbox
- Properties of comm link
 - link established only if processes share a common mailbox
 - a link may be associated with many processes
 - like youre part of a lot of groupchats
 - each pair of processes may share several communication links
 - link may be unidirectional or bi-directional
- Operations
 - create a new mailbox (port)
 - send and receive through mailbox
 - delete a mailbox
- primitives defined as:
 - send(A, message) - send a message to mailbox A
 - receive(A, message) - receive a message from mailbox A

Synchronization

- Message passing may either be **blocking or non-blocking**
- Blocking is synchronous
 - usually associated with direct
 - **Blocking send** - sender is blocked until message is received
 - Sir is checking attendance, call on Evan. Basically Sir is not calling anyone else but Evan.
 - **Blocking receive** - the receiver is blocked until the message is available

- Phone call for example, technically I'm blocked, I can't receive messages while im talking with someone else
- Non-blocking is asynchronous
 - usually associated with indirect
 - Non-blocking send - sender send lang nang send ng message
 - Non-blocking receive - receiver receives:
 - a valid message or a null message
- If both send and receive are blocking, we have a rendezvous
- we can mix and match

Buffering

- Queue of messages attached to a link
 - for example, conencted to yt, loading a video, buffering.
 - you're downloading frames to your buffer, implemented as a queue
1. **Zero capacity** - no messages are queued on a link
 - if you miss my message, you miss my message
 - sender must wait for the receiver (rendezvous)
 2. **Bounded capacity** - finite length of n messages
 - sender must wait if link full
 - back then sa sms, especially nung motorola, the phone doesnt have memory, the memory is in the sim card. you can typically have 50 sms messages and 250 contact list
 - you have a buffer but its limited.
 3. **Unbounded capacity** - infinite length
 - sender never waits
 - think of messenger

IPC Systems - Windows

- Message-passing centric via LPC - local procedure call
 - calling functions in a different process
- only works between processes on the same system
- uses ports like mailboxes
- communication works as follows (marshalling the parameters)
 - client opens handle to subsystem connection port object
 - client sends connection request
 - server creates two private communication ports and return the handle to client
 - client and server use corresponding port hand to send messages or callbacks and listen for replies

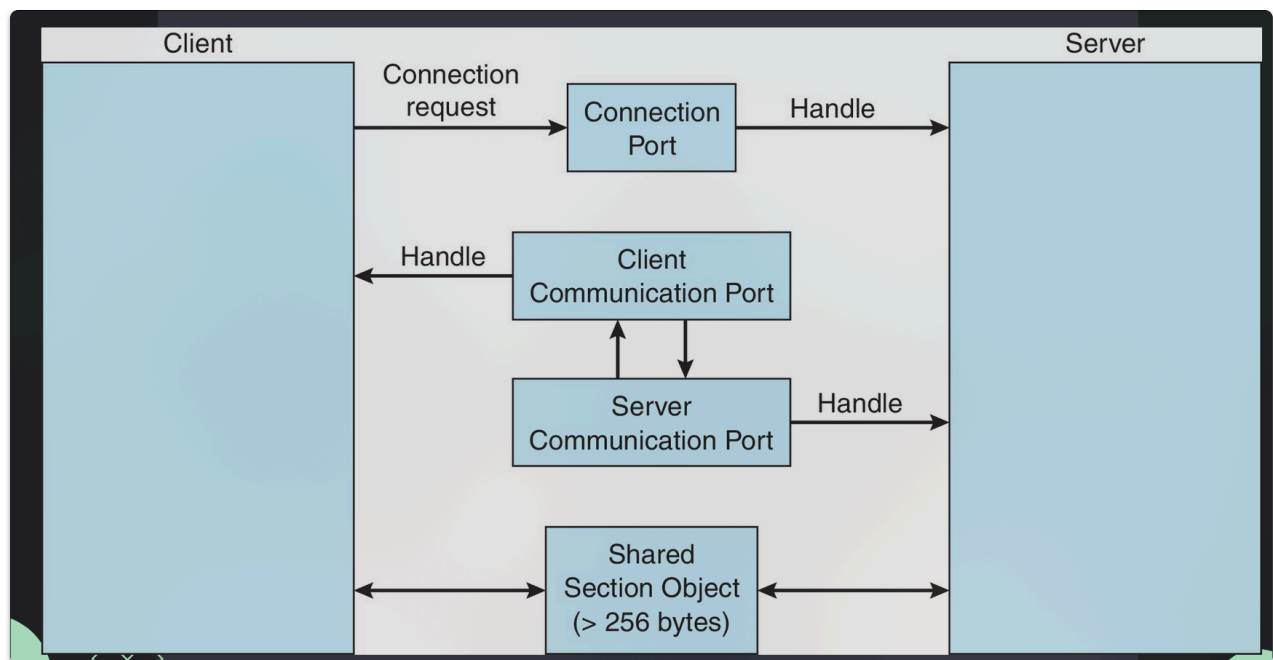


Image 2: IPC system in Windows

Pipes

- common among other OS'
 - conduit, allowing two processes to communicate
1. Ordinary pipes - cannot be accessed from outside the process that create it
 - typically a parent process creates a pipe and uses it to communicate with child process
 2. Named pipes - can be accessed WITHOUT parent-child

Ordinary Pipes

- standard producer-consumer style
- producer writes to one end
- consumer reads from other end
- ordinary pipes are therefore unidirectional
- require parent-child relationship
- Windows calls these anonymous pipes

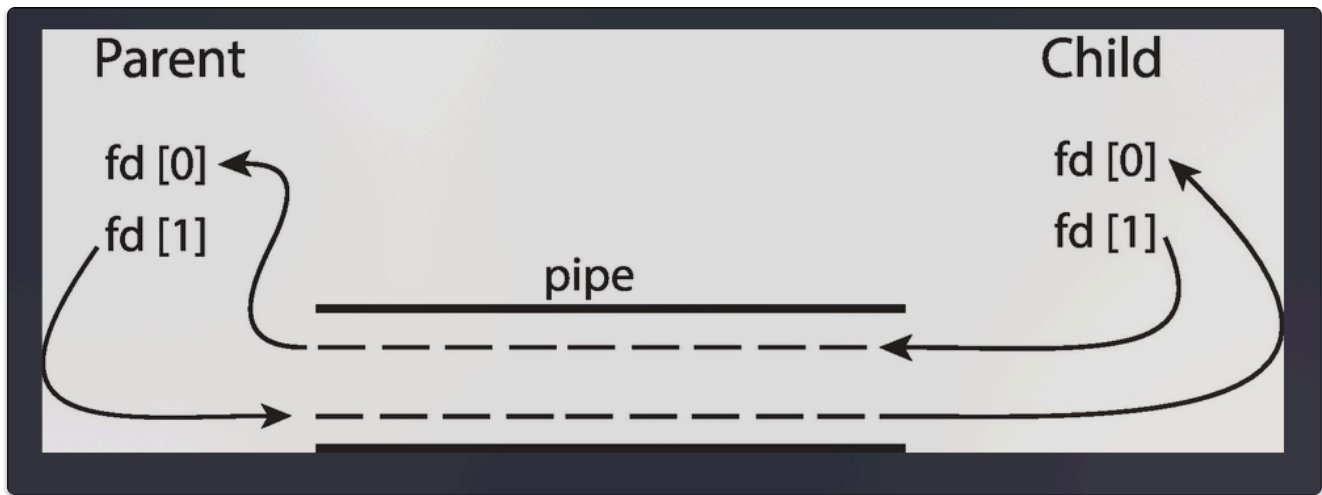


Image 3: Ordinary pipes in windows

Named Pipes

- more memory consumed than ordinary pipes
- communication is bi-directional
- no parent-child relationship necessary
- several processes can use the named pipe for comms
- provided on both UNIX and Windows systems

Client Server Systems

- Sockets
- or Remote Procedure Calls

Sockets

- endpoint for communication
- concatenation of IP add. and port- a number included at start of message packet to differentiate network services on a host
- all ports below 1024 are well known used for standard services
- Special IP address 127.0.0.1 (loopback) to refer to system on which process is running

Connection Oriented - TCP

Connectionless - UDP

Multicast Socket class - data can be sent to multiple recipients

Remote Procedure Calls

- calling a function in a different computer
- Stubs - client-side proxy for the actual procedure on the server
- Client-side stub locates server and marshalls parameters
- Server-side stub receives this message, unpacks the marshalled parameters and performs procedure on server

- On Windows, stub code compile from specification written in Microsoft Interface Definition Language (MIDL)